```
# First layer is downshaping the 28 x 28 image to a 512 tensor vector, relu is max(0, input)
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)));

# Now, add the last layer, downshaping the output of the previous layer to 10 (i.e. 0-9) 'digits'.
# Remember: The output of this layer is merely a probablistic distribution, hence "softmax".
network.add(layers.Dense(10, activation='softmax'))

## let's see how DL network looks like
network.summary()
```

The output is:

```
Instructions for updating:
Co-locations handled automatically by placer.
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output shape              Param #
=================================================================
dense_1 (Dense)              (None, 512)               401920
_________________________________________________________________
dense_2 (Dense)              (None, 10)                5130
=================================================================
Total params: 407,050
Trainable params: 407,050
Non-trainable params: 0_________________________________________________________________
```

The compilation of the network can be done as follows:

```
# Let's compile the network
# Well: compile here means setting up the back propagation, defining the loss function and determining how the model is working
## Optimiser is to reduce the loss function at every step, gradient descent, reducing d(loss_fn)/d (lthe ayer's weight)
# As it's a categocial classification, picking up usual cat_crossent as loss method
# How my network is converging? Lets measure 'accuracy'
network.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy']);
```

For data preparation, here's the code snippet:

```
# Data preparation: To fit to my model

train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype('float32') / 255

# Sort of encoding such that the value remains b/w [-1, to +1]
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

Training can be done as follows:

```
# Now Training is done with 5 steps "epoch" on training data
# Batch size typically power of 2, it's the per batch processing in an epoch
validation_images = train_images[:10000]
actual_train_images = train_images[10000:]
validation_labels = train_labels[:10000]
actual_train_labels = train_labels[10000:]

history = network.fit(actual_train_images, actual_train_labels,
                      epochs=5,
                      batch_size=128,
                      validation_data=(validation_images, validation_labels))
```

The output is:

```
Train on 50000 samples, validate on 10000 samples
Epoch 1/5
50000/50000 [==============================] - 9s 178us/step - loss: 0.2801 - acc: 0.9191 - val_loss: 0.1498 - val_acc: 0.9578
Epoch 2/5
50000/50000 [==============================] - 5s 96us/step - loss: 0.1170 - acc: 0.9656 - val_loss: 0.1070 - val_acc: 0.9675
Epoch 3/5
50000/50000 [==============================] - 5s 100us/step - loss: 0.0773 - acc: 0.9770 - val_loss: 0.0858 - val_acc: 0.9739
Epoch 4/5
50000/50000 [==============================] - 5s 99us/step - loss: 0.0555 - acc: 0.9828 - val_loss: 0.0879 - val_acc: 0.9734
Epoch 5/5
50000/50000 [==============================] - 5s 94us/step - loss: 0.0414 - acc: 0.9874 - val_loss: 0.0812 - val_acc: 0.9767
```

Note: As you can see, the loss function *(val_loss)* is reducing at every step whereas the accuracy *(val_acc)* is increasing at every step. That's the effectiveness of the training.

Here is the code snippet for training vs validation plotting:

```
# history object, output of fit methof
his_dict = history.history
print (his_dict.keys())
```